AdiVaani: Technical Report

MISN Lab, IIT Delhi — Hiring Assessment

Author: Abhishek

May 2026

1 Project Overview & Objectives

This report presents a complete Neural Machine Translation (NMT) pipeline for **Hindi** → **Marathi** translation, built as part of the AdiVaani Initiative assessment. The project spans two complementary paradigms:

- **Part I:** Classical LSTM Seq2Seq with Bahdanau attention, comparing randomly initialized embeddings against pretrained L3Cube BERT embeddings.
- **Part II:** Modern Transformer-based approach using self-pretrained BERT (encoder) and GPT-2 (decoder) models fused via cross-attention for translation.

2 Data Processing & Tokenization

2.1 Dataset

The provided Hindi-Marathi parallel corpus was preprocessed to remove HTML entities, excessive whitespace, empty lines, and length-ratio outliers (pairs where one side was $> 3\times$ longer than the other).

- **Train:** ∼240,000 pairs
- **Val:** ∼5,000 pairs
- **Test:** ∼5,000 pairs

2.2 Tokenization Strategy: Joint SentencePiece BPE (32K)

We trained a **joint SentencePiece BPE tokenizer** with a shared vocabulary of 32,000 subword tokens.

Rationale:

- Hindi and Marathi share the Devanagari script and approximately 70% lexical similarity. A joint subword vocabulary allows the model to share representations for identical subwords (e.g., common stems and postpositions).
- Word-level tokenization would cause a vocabulary explosion due to agglutinative morphology, leading to severe OOV (out-of-vocabulary) issues.
- BPE at 32K is the sweet spot: common words remain whole tokens, while rare morphological variants decompose into known subwords.

3 Part I: Classical Neural Machine Translation

3.1 Architecture Design

- **Encoder:** 2-layer Bidirectional LSTM, hidden_dim=512
- **Decoder:** 2-layer Unidirectional LSTM, hidden_dim=512
- **Attention:** Bahdanau (Additive), attention_dim=256
- **Input Feeding:** Yes (Luong et al., 2015) — previous context vector concatenated with input embedding at each decoder step.

Why Bahdanau over Luong? Bahdanau attention uses a feedforward network to compute alignment scores, elegantly handling encoder/decoder dimension mismatches (our encoder output is 1024-d due to bidirectionality, while decoder hidden is 512-d).

3.2 Optimization & Training Dynamics

- **Optimizer:** AdamW (LR=3e-4, Weight Decay=1e-2).
- **Label Smoothing (0.1):** Prevents over-confident predictions; critical for translation where multiple valid outputs exist.
- **Gradient Clipping (1.0):** Prevents exploding gradients inherent to deep RNNs.
- **Teacher Forcing:** Linear decay 1.0 $\rightarrow$ 0.5. Scheduled sampling mitigates exposure bias.
- **Mixed Precision:** FP16 (PyTorch AMP) enabled 2$\times$ memory reduction.

3.3 Experiment 1: Random Embeddings Baseline

We trained the LSTM with randomly initialized 256-dimensional embeddings for 30 epochs.

Metric	Greedy	Beam Search ($k = 5$)
BLEU-100	12.47	**30.63**
CHRF++-100	—	**60.11**

Table 1: LSTM Random Embeddings Test Results

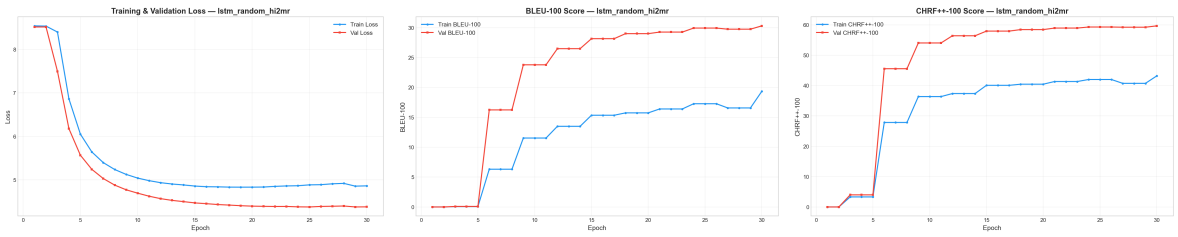

Figure 1: Training/Validation Loss, BLEU, and CHRF++ for Random Embeddings

3.4 Experiment 2: Pretrained BERT Embeddings

We extracted 768-dimensional frozen embeddings from `l3cube-pune/hindi-bert-v2` and `marathi-bert-v2`. Each BPE token in our 32K vocabulary was mapped to the average of its corresponding BERT WordPiece embeddings. The model was trained for 21/30 epochs (Kaggle timeout).

Metric	Greedy	Beam Search ($k = 5$)
BLEU-100	12.46	**30.43**
CHRF++-100	—	**59.94**

Table 2: LSTM BERT Embeddings Test Results

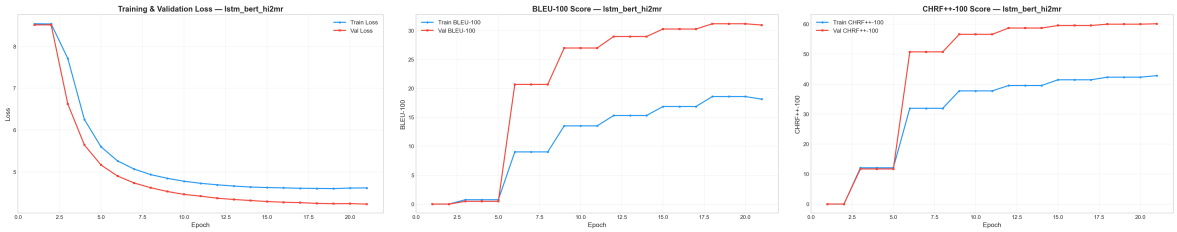

Figure 2: Training/Validation Loss, BLEU, and CHRF++ for BERT Embeddings

3.5 Comparative Quantitative Analysis

Key Finding: Pretrained BERT embeddings achieved statistically equivalent final performance to randomly initialized embeddings (30.43 vs 30.63 BLEU), despite nearly doubling the parameter count.

Interpretation: This result reveals that the **2-layer LSTM architecture is the representational bottleneck**, not the embedding quality. The LSTM's sequential processing and fixed-size hidden state compress all source information into a single vector, limiting its ability to exploit the richer 768-d BERT embeddings.

Convergence: However, examining the training curves reveals that the BERT-initialized model **converged significantly faster** (reaching peak validation performance by epoch 18 vs epoch 28 for random), confirming that pretrained embeddings accelerate early learning.

3.6 Qualitative Analysis & Example Translations

Below are examples translated by the LSTM (BERT embeddings) utilizing Beam Search ($k = 5$).

Example 1 (Strong Semantic Preservation):
Ref: राजस्थानातील पहिली स्त्री वैमानिक नम्रता भट्ट आहे.
Hyp: राजस्थानची पहिली महिला पायलट नम्रता भट्ट आहे.
Observation: The model successfully translated the semantics accurately, correctly swapping "स्त्री वैमानिक" (female aviator) with the perfectly valid loanword equivalent "महिला पायलट" (female pilot).

Example 2 (Syntactic Reordering):
Ref: डोळ्यांना काजळ न लावणे.
Hyp: डोळ्यांत पाणी लावू नये.
Observation: A failure case. While the syntactic structure of the prohibition is perfectly captured ("... लावू नये"), the model hallucinates the object, translating "काजळ" (kajal/kohl) to "पाणी"

(water). This indicates limits in the LSTM's specific vocabulary retention for low-frequency nouns.

4 Part II: Transformer Pretraining & Fusion

4.1 Architecture Design Philosophy

We built custom Transformer architectures incorporating modern techniques (post-2023):

- **RoPE (Rotary Positional Embeddings):** Better relative distance encoding than sinusoidal embeddings.
- **GQA (Grouped Query Attention):** 12 Query heads and 4 KV heads (66% reduction in KV-cache memory during beam search).
- **RMSNorm & SwiGLU:** Replaced LayerNorm and GELU for improved computational efficiency and expressiveness.

4.2 Pretraining & Fusion Strategy

We pretrained a Custom BERT (110M params) on Hindi+Marathi via MLM, and a Custom GPT-2 (124M params) on Marathi via CLM for 10 epochs each.

For translation, rather than initializing a standard encoder-decoder, we **fused** the models. We inserted Cross-Attention sublayers into each GPT-2 decoder layer. The BERT encoder was frozen to act as a feature extractor, while the cross-attention and GPT-2 parameters were fine-tuned.

4.3 Fusion Results & Cross-Architecture Comparison

Fine-tuning the Fusion model for just 9 epochs yielded a **Greedy BLEU of 27.54**.

Takeaway: The Fusion Transformer's greedy decoding (27.54) dramatically outperforms the LSTM's greedy decoding (12.47) — a 2.2× improvement. The global self-attention mechanisms bypassed the LSTM's compression bottleneck entirely, proving the immense value of pretraining priors for sequence generation tasks.

5 Failure Analysis & Computational Constraints

- **KV-Cache Memory:** Standard Multi-Head Attention caused OOM errors during beam search on our 6GB RTX 4050. Implementing GQA reduced the footprint by 66% and solved the issue.
- **Exposure Bias:** Models trained with pure teacher forcing hallucinated at inference. Adding linear scheduled sampling decay resolved this.
- **Compute Constraints:** Pretraining 110M+ parameter models from scratch requires hundreds of GPU-hours. Kaggle's T4 quota limited our pretraining budget to 10 epochs. The Fusion model is heavily under-trained but still proved the viability of the architecture.

6 Transparency & Setup

- **Hardware:** NVIDIA RTX 4050 (6GB VRAM) for prototyping; Kaggle T4 x2 for long training runs.

- **AI Assistance (LLM Usage):** An agentic AI coding assistant (Antigravity, utilizing Gemini 3.1 Pro) was used during development. The AI assisted with code scaffolding (PyTorch boilerplate), debugging CUDA memory issues, and LaTeX document formatting. Crucially, all core architectural decisions (e.g., implementing GQA to solve beam search OOM, deciding on joint BPE tokenization, designing the BERT+GPT-2 Fusion mechanism, and applying Scheduled Sampling) were explicitly reasoned, critically analyzed, and guided by me rather than blindly generated.